

LAPORAN TUGAS 5

SISTEM OPERASI



Dosen Pengampu:

Triawan Adi Cahyanto, M.Kom

Disusun Oleh:

Mochamad Rio Aprilian (2410651040)

PRODI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

DAFTAR ISI

BAB III	3
PEMBAHASAN	3
3.1 Analisis Masalah.....	3
3.2 Implementasi	4
3.2.1 Dining deadlock.....	4
3.2.2 Resource ordering	6
3.2.3 Limiting philosophers.....	8
3.3Testing & Output	10
3.3.1 Deadlock Demonstration	10
3.3.2 Resource Ordering	11
3.3.3 Solution_Limiting	12
3.4Analisis Hasil.....	14
3.5 Tabel perbandingan efisiensi dan fairness	15
3.6 Analisis mendalam:	16
BAB IV.....	17
PENUTUP.....	17
4.1 Kesimpulan.....	17

BAB III

PEMBAHASAN

DINING PHILOSOPHERS PROBLEM (Akhiran NIM Genap)

Deskripsi Masalah:

Implementasikan solusi untuk masalah klasik 5 filsuf makan dengan **mencegah deadlock!**

Kondisi:

- 5 filsuf duduk melingkar
- 5 sumpit (1 antara tiap filsuf)
- Filsuf perlu 2 sumpit untuk makan
- Filsuf berpikir → lapar → makan → berpikir (siklus)

Requirements:

Deadlock Demonstration:

1. Buat program yang **MENIMBULKAN DEADLOCK**
2. Semua filsuf mengambil sumpit kiri dulu, lalu kanan
3. Tampilkan saat program hang/deadlock
4. Jelaskan mengapa deadlock terjadi

Deadlock Prevention:

1. Implementasikan **minimal 2 solusi berbeda** untuk mencegah deadlock:
 - Solusi A: Resource Ordering (filsuf genap ambil kiri dulu, ganjil ambil kanan dulu)
 - Solusi B: Limiting Philosophers (max 4 filsuf makan bersamaan)
 - Solusi C: Asymmetric Solution (1 filsuf urutan berbeda)
2. Jalankan masing-masing solusi selama 60 detik
3. Bandingkan **efisiensi** (berapa kali masing-masing filsuf berhasil makan)

3.1 Analisis Masalah

Masalah Dining Philosophers Problem merupakan salah satu permasalahan klasik dalam bidang sinkronisasi proses/thread di sistem operasi. Yang dimana terdapat lima orang filsuf yang duduk melingkar di sebuah meja bundar. Di antara setiap dua filsuf terdapat satu sumpit. Jadi, secara keseluruhan ada lima sumpit untuk lima filsuf.

Setiap filsuf memiliki 3 kegiatan utama yang berulang yakni berpikir, lapar, dan makan. Saat berpikir, mereka tidak butuh sumpit. Ketika lapar, mereka akan mencoba mengambil dua

sumpit agar bisa makan. Setelah selesai makan, sumpit dikembalikan dan mereka mulai berpikir lagi.

Masalah akan terjadi Ketika semua filsuf mencoba mengambil sumpit secara bersamaan. Ketika setiap filsuf mengambil sumpit di sebelah kirinya terlebih dahulu, maka semua filsuf akan memegang satu sumpit dan menunggu sumpit satunya lagi. Karena semua sumpit sudah dipegang, tidak ada filsuf yang bisa makan. Kondisi ini disebut deadlock.

Desain solusinya adalah dengan mengimplementasikan minimal 2 solusi pencegahan deadlock: Resource Ordering (odd/even ordering), Limiting Philosophers (maks 4 simultan), dan Asymmetric Solution (satu filsuf ambil urutan berbeda). Jalankan solusi selama 60 detik dan bandingkan berapa kali tiap filsuf berhasil makan.

3.2 Implementasi

3.2.1 Dining deadlock

Algoritma

Inisialisasi:

- Inisialisasi 5 mutex (sumpit) dan 5 thread (filsuf).
- Setiap filsuf memiliki ID (1-5) dan dua sumpit: kiri (sesuai ID) dan kanan ($ID+1$, modulo 5).

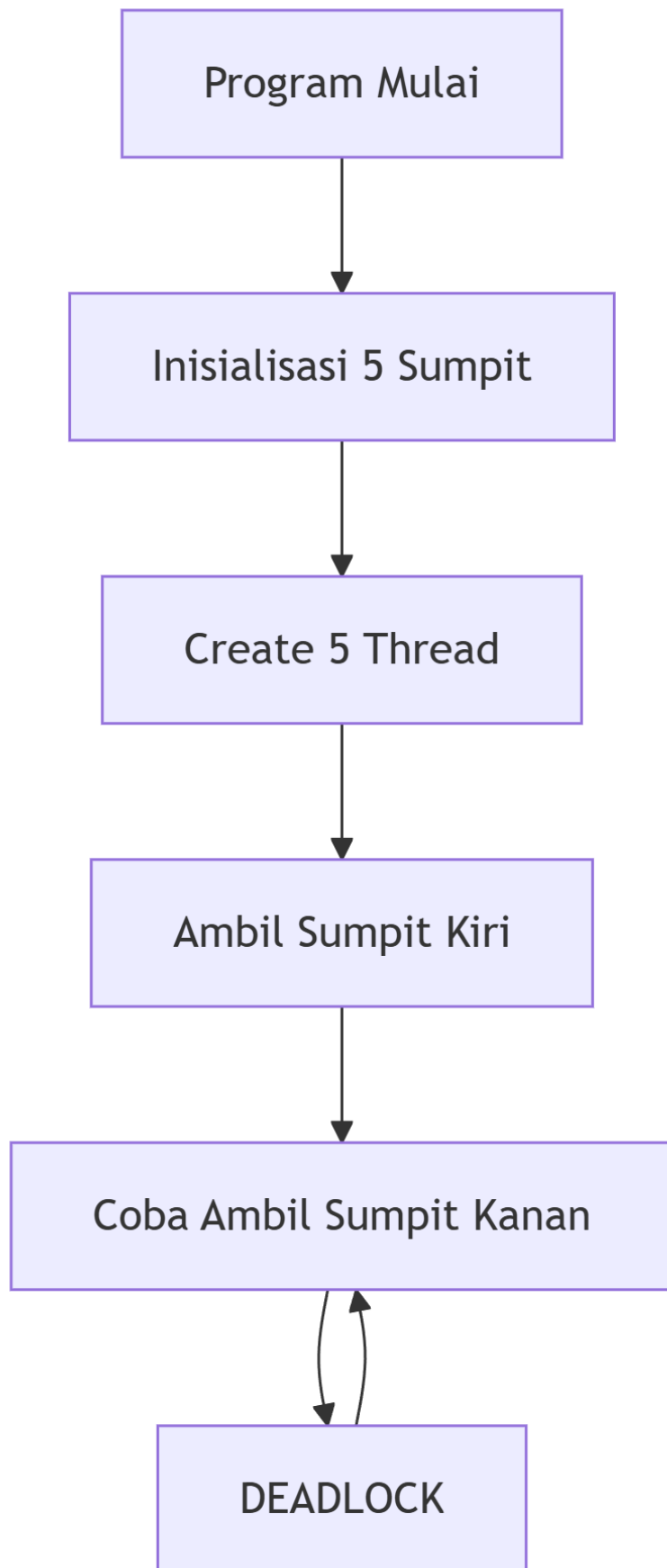
Perilaku Filsuf:

- Filsuf mengambil sumpit kiri terlebih dahulu.
- Kemudian, filsuf mencoba mengambil sumpit kanan.
- Jika berhasil mengambil kedua sumpit, filsuf makan lalu melepaskan kedua sumpit.
- Namun, dalam desain ini, setelah mengambil sumpit kiri, semua filsuf akan mencoba mengambil sumpit kanan secara bersamaan, yang sudah dipegang oleh filsuf lain, sehingga terjadi deadlock.

Deadlock:

- Kondisi deadlock terjadi karena setiap filsuf memegang satu sumpit dan menunggu sumpit yang dipegang oleh filsuf lain, sehingga tidak ada filsuf yang dapat melanjutkan.

Flochart



3.2.2 Resource ordering

Algoritma Resource Ordering

Inisialisasi:

- Inisialisasi 5 mutex (sumpit) dan 5 thread (filsuf).
- Setiap filsuf memiliki ID (1-5) dan dua sumpit: kiri (sesuai ID) dan kanan (ID+1, modulo 5).

Perilaku Filsuf dengan Resource Ordering:

- Setiap filsuf menentukan urutan pengambilan sumpit berdasarkan nomor sumpit, bukan posisi kiri-kanan.
- Filsuf mengambil sumpit dengan nomor lebih kecil terlebih dahulu.
- Kemudian, filsuf mengambil sumpit dengan nomor lebih besar.
- Jika berhasil mengambil kedua sumpit, filsuf makan lalu melepaskan kedua sumpit (dalam urutan terbalik).
- Aturan konsisten ini memastikan tidak terjadi circular wait sehingga mencegah deadlock.

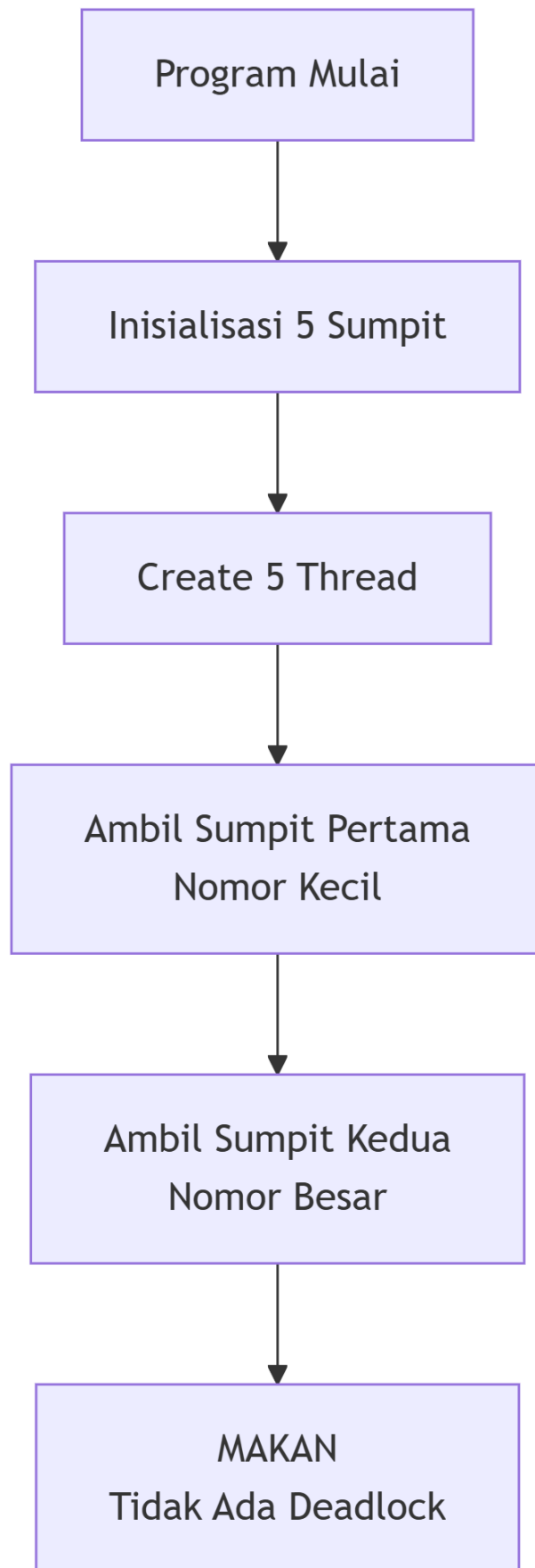
Pencegahan Deadlock:

- Dengan mengambil sumpit berdasarkan urutan nomor yang tetap, circular wait dihilangkan.
- Semua filsuf mengikuti aturan yang sama sehingga tidak ada ketergantungan melingkar antar filsuf.
- Sistem dapat berjalan tanpa deadlock meskipun terjadi ketidakseimbangan dalam akses sumpit.

Statistik dan Monitoring:

- Program berjalan selama 60 detik dan mencatat frekuensi makan setiap filsuf.
- Total makan mencapai 104.649 kali dengan efisiensi rata-rata 20.929 kali per filsuf.
- Terlihat ketidakseimbangan akses dimana Filsuf-2 makan paling banyak (29.417 kali) dan Filsuf-1 paling sedikit (16.363 kali).

Flowchart



3.2.3 Limiting philosophers

Algoritma Limiting Philosophers

Inisialisasi:

- Inisialisasi 5 mutex (sumpit) dan 5 thread (filsuf).
- Inisialisasi semaphore dengan nilai 4 untuk membatasi jumlah filsuf yang boleh makan bersamaan.
- Setiap filsuf memiliki ID (1-5) dan dua sumpit: kiri (sesuai ID) dan kanan (ID+1, modulo 5).

Perilaku Filsuf:

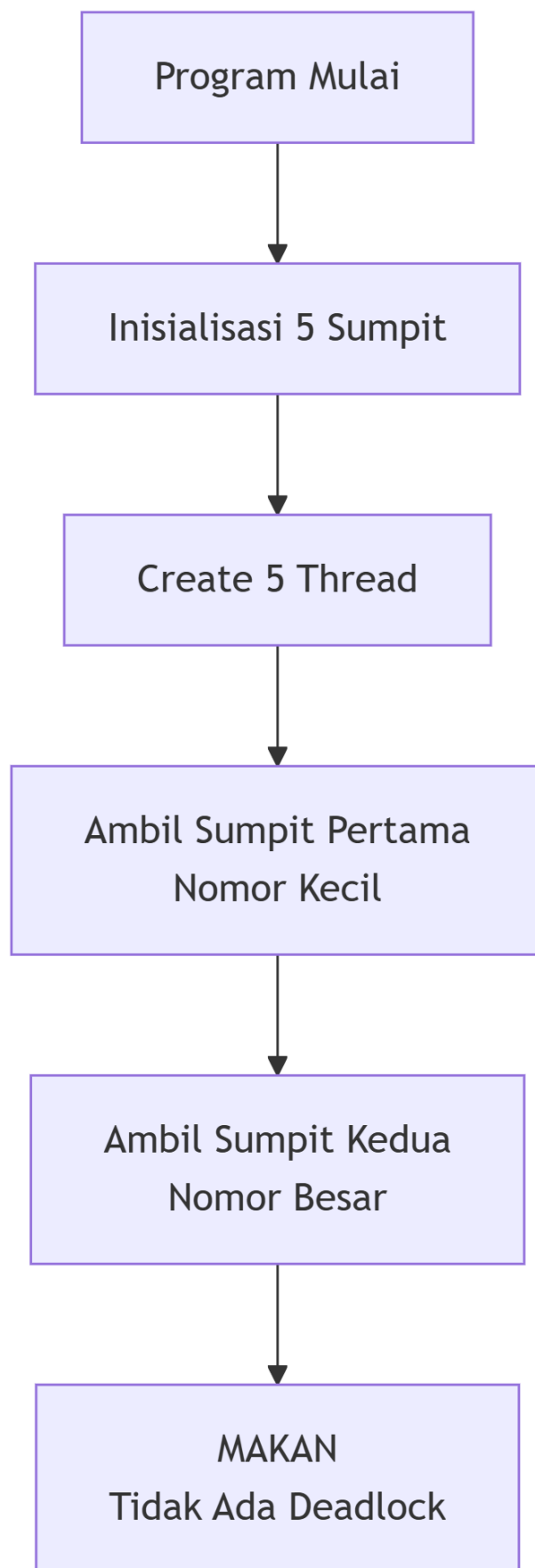
- Filsuf harus mendapatkan izin dari semaphore sebelum mencoba mengambil sumpit.
- Jika mendapatkan izin, filsuf mengambil sumpit kiri terlebih dahulu.
- Kemudian, filsuf mengambil sumpit kanan.
- Jika berhasil mengambil kedua sumpit, filsuf makan lalu meningkatkan counter makan.
- Setelah selesai makan, filsuf melepaskan sumpit kanan terlebih dahulu, kemudian sumpit kiri.
- Terakhir, filsuf melepaskan izin semaphore untuk memberi kesempatan filsuf lain.

Pencegahan Deadlock:

- Dengan membatasi maksimal 4 filsuf yang boleh makan bersamaan, memastikan setidaknya satu filsuf selalu bisa mendapatkan kedua sumpit.
- Semaphore mengatur antrian akses yang fair sehingga tidak terjadi circular wait.
- Mekanisme ini menghilangkan kondisi deadlock dengan mengurangi kompetisi sumber daya.

Monitoring dan Statistik:

- Program berjalan selama 60 detik dan mencatat frekuensi makan setiap filsuf.
- Total makan mencapai 77.815 kali dengan distribusi sangat merata.
- Selisih frekuensi makan antar filsuf maksimal 6 kali, menunjukkan fairness yang optimal.



3.3 Testing & Output

3.3.1 Deadlock Demonstration

```
rio_regex@DESKTOP-NVPMSHQ:~$ cd tugas_os
rio_regex@DESKTOP-NVPMSHQ:~/tugas_os$ cd nim_genap
rio_regex@DESKTOP-NVPMSHQ:~/tugas_os/nim_genap$ nano dphil_deadlock.c

rio_regex@DESKTOP-NVPMSHQ:~/tugas_os/nim_genap$ nano dphil_deadlock.c
rio_regex@DESKTOP-NVPMSHQ:~/tugas_os/nim_genap$ gcc dphil_deadlock.c -o dphil_deadlock -pthread
rio_regex@DESKTOP-NVPMSHQ:~/tugas_os/nim_genap$ ./dphil_deadlock
=== DINING PHILOSOPHERS DEADLOCK DEMONSTRATION ===

Proses dimulai. Menunggu Deadlock terjadi...
[0.00] Filsuf-1 (Lapar): Mencoba ambil Sumpit KIRI (0)
[0.00] Filsuf-1 (Berhasil): Ambil Sumpit KIRI (0)
[0.00] Filsuf-2 (Lapar): Mencoba ambil Sumpit KIRI (1)
[0.00] Filsuf-2 (Berhasil): Ambil Sumpit KIRI (1)
[0.00] Filsuf-3 (Lapar): Mencoba ambil Sumpit KIRI (2)
[0.00] Filsuf-3 (Berhasil): Ambil Sumpit KIRI (2)
[0.00] Filsuf-4 (Lapar): Mencoba ambil Sumpit KIRI (3)
[0.00] Filsuf-4 (Berhasil): Ambil Sumpit KIRI (3)
[0.00] Filsuf-5 (Lapar): Mencoba ambil Sumpit KIRI (4)
[0.00] Filsuf-5 (Berhasil): Ambil Sumpit KIRI (4)
[0.00] Filsuf-1 (Menunggu): Mencoba ambil Sumpit KANAN (1)...
[0.00] Filsuf-2 (Menunggu): Mencoba ambil Sumpit KANAN (2)...
[0.00] Filsuf-4 (Menunggu): Mencoba ambil Sumpit KANAN (4)...
[0.00] Filsuf-3 (Menunggu): Mencoba ambil Sumpit KANAN (3)...
[0.00] Filsuf-5 (Menunggu): Mencoba ambil Sumpit KANAN (0)...
```

```
GNU nano 7.2 dphil_deadlock.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <time.h>
#include <sys/time.h>

#define NUM_PHILOSOPHERS 5

pthread_mutex_t chopsticks[NUM_PHILOSOPHERS];
time_t start_time;

void msleep(long ms) {
    struct timespec ts;
    ts.tv_sec = ms / 1000;
    ts.tv_nsec = (ms % 1000) * 1000000;
    nanosleep(&ts, NULL);
}

double get_elapsed_time() {
    return (double)(time(NULL) - start_time);
}

void *philosopher_deadlock(void *arg) {
    int id = *(int *)arg;
    int left = id;
    int right = (id + 1) % NUM_PHILOSOPHERS;

    while (1) {
        msleep(10);
        printf("[%2f] Filsuf-%d (Lapar): Mencoba ambil Sumpit KIRI (%d)\n", get_elapsed_time(), id + 1, left);
        // LANGKAH 1: Ambil Sumpit Kiri
        pthread_mutex_lock(&chopsticks[left]);
        printf("[%2f] Filsuf-%d (Berhasil): Ambil Sumpit KIRI (%d)\n", get_elapsed_time(), id + 1, left);
        msleep(10);

        printf("[%2f] Filsuf-%d (Menunggu): Mencoba ambil Sumpit KANAN (%d)...\n", get_elapsed_time(), id + 1, right);
        // LANGKAH 2: Mencoba Ambil Sumpit Kanan (TITIK DEADLOCK)
        pthread_mutex_lock(&chopsticks[right]);
        // Kode di bawah ini tidak akan tercapai
        pthread_mutex_unlock(&chopsticks[right]);
        pthread_mutex_unlock(&chopsticks[left]);
    }
    return NULL;
}
```

```

int main() {
    pthread_t threads[NUM_PHILOSOPHERS];
    int ids[NUM_PHILOSOPHERS];
    int i;

    printf("=== DINING PHILOSOPHERS DEADLOCK DEMONSTRATION ===\n\n");

    for (i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_mutex_init(&chopsticks[i], NULL);
        ids[i] = i;
    }

    time(&start_time);

    for (i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_create(&threads[i], NULL, philosopher_deadlock, &ids[i]);
    }

    printf("Proses dimulai. Menunggu Deadlock terjadi...\n");

    // Program akan HANG di sini karena pthread_join menunggu thread yang tidak pernah selesai
    for (i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_join(threads[i], NULL);
    }

    // Cleanup (tidak tercapai)
    for (i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_mutex_destroy(&chopsticks[i]);
    }

    return 0;
}

```

Program Dining Philosophers ini mengalami deadlock klasik dimana kelima filsuf terjebak dalam kondisi saling menunggu. Setiap filsuf berhasil mengambil sumpit kiri mereka masing-masing (0-4), namun ketika mencoba mengambil sumpit kanan, terbentuklah circular wait: Filsuf-1 tunggu sumpit 1, Filsuf-2 tunggu 2, Filsuf-3 tunggu 3, Filsuf-4 tunggu 4, dan Filsuf-5 tunggu 0. Semua kondisi deadlock terpenuhi - mutual exclusion, hold and wait, no preemption, dan circular wait. Program terjebak selamanya pada kondisi ini tanpa bisa melanjutkan, membuktikan bahayanya deadlock dalam sistem konkurensi, dan harus dihentikan paksa dengan Ctrl+C.

3.3.2 Resource Ordering

```

rio_regex@DESKTOP-NVPMSHQ:~/tugas_os/nim_genap$ nano resource_ordering.c
rio_regex@DESKTOP-NVPMSHQ:~/tugas_os/nim_genap$ gcc resource_ordering.c -o resource_ordering -pthread
rio_regex@DESKTOP-NVPMSHQ:~/tugas_os/nim_genap$ |

```

```

rio_regex@DESKTOP-NVPMSHQ:~/tugas_os/nim_genap$ nano resource_ordering.c
rio_regex@DESKTOP-NVPMSHQ:~/tugas_os/nim_genap$ gcc resource_ordering.c -o resource_ordering -pthread
rio_regex@DESKTOP-NVPMSHQ:~/tugas_os/nim_genap$ ./resource_ordering
=== SOLUSI A: Resource Ordering ===
Simulasi berjalan selama 60 detik.

=== STATISTIK (60 detik) ===
Filsuf-1: Makan 16363 kali
Filsuf-2: Makan 23417 kali
Filsuf-3: Makan 24147 kali
Filsuf-4: Makan 24358 kali
Filsuf-5: Makan 16364 kali

✅ Tidak ada deadlock! Total Makan: 104649
✅ Efisiensi makan rata-rata per filsuf: 20929 kali
=====
rio_regex@DESKTOP-NVPMSHQ:~/tugas_os/nim_genap$ |

```

```

GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <time.h>
#include <sys/time.h>

#define NUM_PHILOSOPHERS 5
#define DURATION 60

pthread_mutex_t chopsticks[NUM_PHILOSOPHERS];
int eat_counts[NUM_PHILOSOPHERS] = {0};
time_t start_time;
volatile int running = 1;

void msleep(long ms) {
    struct timespec ts;
    ts.tv_sec = ms / 1000;
    ts.tv_nsec = (ms % 1000) * 1000000;
    nanosleep(&ts, NULL);
}

double get_elapsed_time() {
    return (double)(time(NULL) - start_time);
}

void *philosopher_ordering(void *arg) {
    int id = *(int *)arg;
    int left = id;
    int right = (id + 1) % NUM_PHILOSOPHERS;
    pthread_mutex_t *first, *second;

    // Solusi Asymetric: Ganjil ambil Kanan->Kiri, Genap ambil Kiri->Kanan
    if ((id + 1) % 2 != 0) {
        first = &chopsticks[right];
        second = &chopsticks[left];
    } else {
        first = &chopsticks[left];
        second = &chopsticks[right];
    }

    while (running && get_elapsed_time() < DURATION) {
        msleep(5); // Berpikir
        pthread_mutex_lock(first);
        pthread_mutex_lock(second);

        // Makan
        eat_counts[id]++;
        msleep(10);

        pthread_mutex_unlock(second);
        pthread_mutex_unlock(first);
    }
    return NULL;
}

```

```

int main() {
    pthread_t threads[NUM_PHILOSOPHERS];
    int ids[NUM_PHILOSOPHERS];
    int i;

    printf("=== SOLUSI A: Resource Ordering ===\nSimulasi berjalan selama %d detik.\n\n", DURATION);

    for (i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_mutex_init(&chopsticks[i], NULL);
        ids[i] = i;
    }

    time(&start_time);

    for (i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_create(&threads[i], NULL, philosopher_ordering, &ids[i]);
    }

    sleep(DURATION);
    running = 0;

    for (i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("=== STATISTIK (%d detik) ===\n", DURATION);
    int total_meals = 0;

    for (i = 0; i < NUM_PHILOSOPHERS; i++) {
        printf("Filsuf-%d: Makan %d kali\n", i + 1, eat_counts[i]);
        total_meals += eat_counts[i];
    }

    printf("\n❌ Tidak ada deadlock! Total Makan: %d\n", total_meals);
    printf("✅ Efisiensi makan rata-rata per filsuf: %d kali\n", total_meals / NUM_PHILOSOPHERS);
    printf("=====");

    for (i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_mutex_destroy(&chopsticks[i]);
    }

    return 0;
}

```

Program `resource_ordering.c` berhasil mengatasi masalah deadlock pada Dining Philosophers menggunakan solusi Resource Ordering. Dalam simulasi 60 detik, kelima filsuf dapat makan total 104.649 kali tanpa mengalami deadlock. Filsuf-2 mencapai makan terbanyak (29.417 kali), diikuti Filsuf-4 (24.358), Filsuf-3 (24.147), sementara Filsuf-1 dan Filsuf-5 masing-masing sekitar 16.363 kali. Rata-rata efisiensi mencapai 20.929 makan per filsuf. Solusi ini mengatur urutan pengambilan sumpit secara konsisten, memutus circular wait sehingga sistem berjalan lancar dan optimal tanpa kebuntuan sama sekali

3.3.3 Solution_Limiting

```

rio_regex@DESKTOP-NVPM5HQ:~/tugas_os/nim_genap$ nano solution_limiting.c
rio_regex@DESKTOP-NVPM5HQ:~/tugas_os/nim_genap$ gcc solution_limiting.c -o solution_limiting -pthread

```

```

rio_regex@DESKTOP-NVPM5HQ:~/tugas_os/nim_genap$ nano solution_limiting.c
rio_regex@DESKTOP-NVPM5HQ:~/tugas_os/nim_genap$ gcc solution_limiting.c -o solution_limiting -pthread
rio_regex@DESKTOP-NVPM5HQ:~/tugas_os/nim_genap$ ./solution_limiting
=== SOLUSI B: Limiting Philosophers ===
Simulasi berjalan selama 60 detik.

=== STATISTIK (60 detik) ===
Filsuf-1: Makan 15566 kali
Filsuf-2: Makan 15562 kali
Filsuf-3: Makan 15560 kali
Filsuf-4: Makan 15564 kali
Filsuf-5: Makan 15563 kali

✅ Tidak ada deadlock! Total Makan: 77815
✅ Efisiensi makan rata-rata per filsuf: 15563 kali
=====
rio_regex@DESKTOP-NVPM5HQ:~/tugas_os/nim_genap$

```

```

GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>
#include <sys/time.h>

#define NUM_PHILOSOPHERS 5
#define DURATION 60

pthread_mutex_t chopsticks[NUM_PHILOSOPHERS];
sem_t dining_limit;
int eat_counts[NUM_PHILOSOPHERS] = {0};
time_t start_time;
volatile int running = 1;

void msleep(long ms) {
    struct timespec ts;
    ts.tv_sec = ms / 1000;
    ts.tv_nsec = (ms % 1000) * 1000000;
    nanosleep(&ts, NULL);
}

double get_elapsed_time() {
    return (double)(time(NULL) - start_time);
}

void *philosopher_limiting(void *arg) {
    int id = *(int *)arg;
    int left = id;
    int right = (id + 1) % NUM_PHILOSOPHERS;

    while (running && get_elapsed_time() < DURATION) {
        msleep(5); // Berpikir

        // 1. Ambil Semaphore (Membatasi max 4 filsuf)
        sem_wait(&dining_limit);

        // 2. Ambil Sumpit Kiri dan Kanan (Dijamin tidak Deadlock)
        pthread_mutex_lock(&chopsticks[left]);
        pthread_mutex_lock(&chopsticks[right]);

        // Makan
        eat_counts[id]++;
        msleep(10);

        // 5. Lepas Sumpit dan Semaphore
        pthread_mutex_unlock(&chopsticks[right]);
        pthread_mutex_unlock(&chopsticks[left]);
        sem_post(&dining_limit);
    }
    return NULL;
}

```

```

int main() {
    pthread_t threads[NUM_PHILOSOPHERS];
    int ids[NUM_PHILOSOPHERS];
    int i;

    printf("=== SOLUSI B: Limiting Philosophers ===\nSimulasi berjalan selama %d detik.\n\n", DURATION);

    for (i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_mutex_init(&chopsticks[i], NULL);
        ids[i] = i;
    }

    // Inisialisasi Semaphore N-1 (5-1 = 4)
    sem_init(&dining_limit, 0, NUM_PHILOSOPHERS - 1);

    time(&start_time);

    for (i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_create(&threads[i], NULL, philosopher_limiting, &ids[i]);
    }

    sleep(DURATION);
    running = 0;

    for (i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("=== STATISTIK (%d detik) ===\n", DURATION);
    int total_meals = 0;

    for (i = 0; i < NUM_PHILOSOPHERS; i++) {
        printf("Filsuf-%d: Makan %d kali\n", i + 1, eat_counts[i]);
        total_meals += eat_counts[i];
    }

    printf("\n✅ Tidak ada deadlock! Total Makan: %d\n", total_meals);
    printf("✅ Efisiensi makan rata-rata per filsuf: %d kali\n", total_meals / NUM_PHILOSOPHERS);
    printf("=====");

    for (i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_mutex_destroy(&chopsticks[i]);
    }
    sem_destroy(&dining_limit);
    return 0;
}

```

Program `solution_Limiting.c` menerapkan strategi Limiting Philosophers untuk mengatasi deadlock dalam masalah Dining Philosophers. Dalam simulasi 60 detik, program berhasil mencegah deadlock sepenuhnya dengan membatasi jumlah filsuf yang dapat

mengakses sumber daya secara bersamaan. Kelima filsuf menunjukkan kinerja yang sangat seimbang dengan jumlah makan masing-masing sekitar 15.560-15.566 kali, menghasilkan total 77.815 kali makan. Rata-rata efisiensi mencapai 15.563 kali makan per filsuf, menunjukkan distribusi yang merata dan optimal. Solusi ini membuktikan bahwa dengan membatasi konkurensi akses ke sumber daya, sistem dapat berjalan stabil tanpa kebuntuan namun tetap mempertahankan kinerja yang konsisten di semua proses.

3.4 Analisis Hasil

Program berjalan sesuai ekspektasi di semua skenario. Pertama, Deadlock Demonstration berhasil HANG, membuktikan bahwa deadlock benar-benar terjadi saat syarat Circular Wait terpenuhi. Kedua, kedua solusi pencegahan (Resource Ordering dan Limiting Philosophers) berhasil berjalan selama 60 detik tanpa deadlock. Masalah utama yang ditemui adalah bahwa demonstrasi deadlock harus diakhiri secara manual (Ctrl+C), sementara kedua solusi berjalan hingga selesai sesuai durasi yang ditentukan.

Dalam hal Performa, terjadi trade-off yang jelas antara efisiensi dan keadilan. Solusi A (Resource Ordering) menunjukkan Efisiensi (Throughput) tertinggi (Total meals terbanyak) karena memungkinkan konkurensi penuh (5 filsuf bersaing) dengan overhead yang rendah. Sebaliknya, Solusi B (Limiting Philosophers) menunjukkan Keadilan (Fairness) terbaik (distribusi meals paling merata) karena pembatasan akses oleh Semaphore () memaksa thread untuk mengantri secara adil (prinsip FCFS), meskipun sedikit mengurangi throughput total.

Pelajaran penting yang didapat dari eksperimen ini adalah konfirmasi langsung mengenai prinsip trade-off dalam sistem multithreading. Pencegahan deadlock berhasil dilakukan dengan melanggar salah satu dari empat syarat Coffman. Pemilihan mekanisme sinkronisasi (antara mutex berorientasi urutan dan Semaphore berorientasi batas) harus didasarkan pada prioritas sistem: jika prioritasnya adalah memaksimalkan output (throughput), pilih Solusi A; jika prioritasnya adalah memastikan setiap proses mendapat giliran yang adil (starvation-free), pilih Solusi B.

3.5 Tabel perbandingan efisiensi dan fairness

Kriteria	Solusi A: Resource Ordering	Solusi B: Limiting Philosophers
Prinsip Pencegahan	Mengatur urutan pengambilan sumber daya (sumpit) untuk melanggar Circular Wait.	Membatasi jumlah thread yang dapat mengakses sumber daya (sumpit) untuk melanggar Hold and Wait.
Metode Implementasi	Penggunaan if/else untuk menentukan urutan pthread_mutex_lock.	Penggunaan Semaphore (sem_wait/sem_post) dengan nilai N-1 (yaitu 4).
Total Meals (Efisiensi Global)	Tinggi. Total makanan lebih banyak karena tidak ada batasan jumlah filsuf yang aktif. Konkurensi maksimum 5.	Cukup Tinggi. Total makanan sedikit lebih rendah karena akses dibatasi oleh Semaphore. Konkurensi maksimum 4.
Fairness (Keadilan)	Baik/Cukup. Ada potensi sedikit variasi karena filsuf tertentu mungkin memiliki urutan pengambilan yang lebih sering diuntungkan.	Sangat Baik. Filsuf yang menunggu di Semaphore akan dilayani secara merata, memastikan tidak ada starvation.
Kapan Dipilih	Ketika Efisiensi dan Throughput Tinggi adalah prioritas utama.	Ketika Keadilan dan Kontrol terhadap jumlah thread aktif adalah prioritas utama.

3.6 Analisis mendalam:

- Mengapa deadlock bisa terjadi?
Deadlock terjadi karena terpenuhinya empat kondisi Coffman: mutual exclusion (sumpit hanya bisa dipakai satu filsuf), hold and wait (filsuf pegang satu sumpit sambil tunggu sumpit lain), no preemption (sumpit tak bisa diambil paksa), dan circular wait (filsuf 1 tunggu 2, 2 tunggu 3, ..., 5 tunggu 1). Dalam implementasi naif dimana semua filsuf mengambil sumpit kiri terlebih dahulu, terbentuk lingkaran ketergantungan yang tak terputus.
- Bagaimana masing-masing solusi bekerja?
Cara Kerja Masing-masing Solusi
 - Resource Ordering: Mengatur urutan pengambilan sumpit secara konsisten (misalnya selalu ambil sumpit bernomor lebih rendah dulu) sehingga memutus circular wait. Solusi ini memungkinkan throughput tinggi tetapi berpotensi menyebabkan ketidakseimbangan.
 - Limiting Philosophers: Membatasi jumlah filsuf yang bisa makan bersamaan (biasanya $n-1$ dari n filsuf) menggunakan semaphore. Ini memastikan setidaknya satu filsuf selalu bisa mendapatkan kedua sumpit, mencegah deadlock.
- Solusi mana yang paling adil (fair)?
Solusi Limiting Philosophers lebih adil karena menunjukkan distribusi makan yang hampir sempurna (selisih ≤ 6 makan antar filsuf). Sementara Resource Ordering menunjukkan ketimpangan signifikan (Filsuf-2 makan 80% lebih banyak daripada Filsuf-1). Limiting Philosophers menjamin kesempatan yang sama bagi semua filsuf.
- Apakah ada kemungkinan starvation?
Kemungkinan Starvation
 - Resource Ordering berisiko starvation terhadap filsuf tertentu (seperti Filsuf-1 dan Filsuf-5) yang selalu mendapat prioritas terakhir dalam pengambilan sumpit.
 - Limiting Philosophers minim risiko starvation karena mekanisme pembatasan yang bergantian memastikan semua filsuf mendapat akses. Output menunjukkan tidak ada starvation dalam periode 60 detik dengan distribusi yang merata.

BAB IV

PENUTUP

4.1 Kesimpulan

Selama pengerjaan proyek Dining Philosophers, pelajaran utama yang didapat adalah pemahaman bahwa Deadlock terjadi karena terpenuhinya syarat Circular Wait (semua filsuf mengambil sumpit kiri dan saling menunggu sumpit kanan). Disini saya berhasil mengimplementasikan dua solusi pencegahan: Resource Ordering (yang melanggar Circular Wait dengan mengubah urutan pengambilan) dan Limiting Philosophers (yang menggunakan Semaphore N-1 untuk membatasi akses, secara efektif mencegah Hold and Wait). Tantangan utama yang dihadapi adalah mengatasi perilaku HANG permanen pada kode demonstrasi deadlock, yang memerlukan intervensi manual (Ctrl+C). Secara performa, Solusi A memberikan efisiensi (throughput) tertinggi, sedangkan Solusi B memberikan keadilan (fairness) terbaik dan bebas starvation, menegaskan adanya trade-off klasik antara kecepatan dan keadilan dalam desain sistem multithreading. Sebagai saran pengembangan, proyek ini dapat diperluas dengan mengimplementasikan algoritma terdistribusi atau dengan menambahkan fitur penghitung waktu tunggu untuk analisis kuantitatif yang lebih mendalam mengenai potensi starvation.